

A Web Platform for Generating Optimization-Ready V2V/V2I Connectivity Data from SUMO Vehicular Scenarios

Edison Xavier Espinel Sevilla ^{1,*} 

¹ Escuela Politécnica Nacional, Quito, Ecuador; edison.espinel@epn.edu.ec

Abstract

Roadside-unit (RSU) deployment models need data that tell which vehicles can reach each candidate RSU, in which mobility snapshot, and with how many hops. Producing these data is not trivial: it means combining traffic mobility, urban geometry, direct-link evaluation, graph processing, and optimization-oriented data structures. This paper presents a web platform that turns a geographical area and a SUMO mobility scenario into an optimization-ready connectivity dataset. The backend integrates OpenStreetMap and SUMO tools, filters candidate RSU positions, evaluates binary V2V and V2I links from communication range and building obstruction, and computes cumulative and minimum-hop connectivity through a matrix recurrence. A formal proposition shows that the recurrence finds every vehicle–RSU path within a given hop limit, and first-arrival matrices identify its minimum hop count. A Streamlit frontend supports configuration, execution, visualization, and export. On the Historic Centre of Quito, varying the minimum junction degree and the clustering radius keeps between 177 and 414 of 1211 eligible junctions as candidates (about 66% to 85% reduction). The workflow separates mobility simulation from infrastructure optimization, so the same dataset can feed different exact or heuristic deployment models.

Keywords: VANET; V2V connectivity; V2I connectivity; SUMO; multi-hop connectivity; roadside units; preprocessing; facility location; operations research

1. Introduction

Vehicular ad hoc networks (VANETs) enable direct vehicle-to-vehicle (V2V) communication and vehicle-to-infrastructure (V2I) communication with fixed roadside units (RSUs), usually installed at intersections. Where these units are placed affects coverage, path length, infrastructure cost, and network performance, so RSU deployment is often stated as a facility-location problem and solved with exact, heuristic, or metaheuristic methods.

These models need connectivity information *before* the optimization starts: which vehicles can reach each candidate RSU in each snapshot, and the minimum number of relay hops. Producing it is not a minor step, as it combines traffic simulation, vehicle-position extraction, urban geometry, range checks, obstacle detection, graph processing, and conversion into the structures a solver accepts. Yet many deployment studies treat these data as already available. The software side is similar: SUMO and Python libraries are usually glued together with ad-hoc scripts, which hurts reproducibility and reuse across formulations.

This work fills that gap with a web-based preprocessing layer that follows the sequence

urban area → SUMO snapshots → candidate RSUs → V2V/V2I matrices
→ minimum-hop tuples → optimization input.

Published:

Copyright: © 2026 by the author.

Submitted to *Eng. Proc.* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

Its main output is not just a set of connectivity matrices, but a traceable dataset that keeps the relation between geographical positions, SUMO identifiers, matrix indices, snapshots, hop counts, and optimization identifiers. The contributions are: (i) a web platform that integrates a Streamlit frontend with a Python/SUMO backend in a reproducible workflow; (ii) automatic generation of direct V2V and V2I matrices under a binary communication-range and line-of-sight (LoS) model; (iii) a configurable two-stage method that reduces the candidate RSU set using junction degree and spatial clustering; (iv) computation of cumulative and minimum-hop vehicle–RSU matrices; (v) export of optimization-ready dense structures and sparse tuples, including an artificial no-service option per vehicle; (vi) a formal correctness result for the recurrence and the minimum-hop representation; and (vii) a sensitivity analysis of how the minimum degree and clustering radius change the number of retained candidates. The optimization model that picks the final deployment is out of scope: the goal is to generate its input automatically and independently of any formulation.

2. Related Work

2.1. RSU Deployment and Facility-Location Models

RSU deployment has been studied through covering, median, routing, and stochastic formulations. Maximal-covering models try to cover the largest demand with a limited number of RSUs; set-covering models minimize how many units are needed to satisfy coverage; and median-type models reduce the average distance or hop count. Mobility-aware formulations extend these ideas over several traffic snapshots. The stochastic model in [1] uses scenario-dependent connectivity tuples and adds an artificial disconnected RSU as a no-service alternative that keeps the problem feasible when a vehicle cannot, or should not, be assigned to a deployed real RSU.

Despite their differences, all of these formulations rely on a common data layer: candidate RSUs, vehicle sets, mobility scenarios, and feasible vehicle–RSU relationships. This work focuses on producing exactly that layer. **[TO DO: Complete this subsection with recent studies on mobility-aware RSU placement, multi-hop access, and exact or heuristic deployment. Every added reference must be cited in the text.]**

2.2. SUMO-Based Platforms and Web Interfaces

SUMO is an open-source microscopic traffic simulator with utilities to import OSM networks, generate vehicle trips, run simulations, and export floating-car data (FCD) [2, 3]. Streamlit and Folium support interactive Python web interfaces and geographical visualization [4,5]. Their individual capabilities are not, by themselves, the contribution of this work. The contribution is their orchestration together with candidate filtering, geometric direct-link evaluation, multi-hop processing, identifier traceability, and export to optimization models.

Existing SUMO-based applications often focus on simulation control or visualization. This platform instead treats simulation as the first stage of a data-generation process whose final output can be consumed by facility-location and network-design formulations. **[TO DO: Add a short comparison with the closest scientific platforms, considering mobility simulation, obstacle-aware V2V/V2I generation, multi-hop processing, web interaction, optimization export, and reproducibility.]**

3. Web Platform and Connectivity-Data Generation

3.1. Platform Architecture and Workflow

The platform follows a simplified model–view–controller organization. The Streamlit frontend captures the area and the parameters, launches the workflow, and displays maps,

matrices, and summary indicators, while the Python backend prepares the SUMO scenario, extracts snapshots, filters candidate RSUs, evaluates direct links, computes multi-hop reachability, and exports the dataset; an orchestrator coordinates the modules and stores each run's configuration. As shown in Figure 1, OSM data become a SUMO network and building polygons through `netconvert` and `polyconvert`, `randomTrips.py` generates the demand, and the simulation produces the FCD snapshots that feed the connectivity engine.

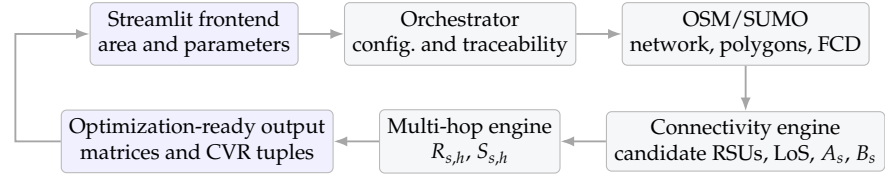


Figure 1. Platform architecture and end-to-end data-generation workflow.

The configurable parameters are grouped into mobility settings (bounding box, simulation duration, vehicle generation, snapshot interval, and random seed), candidate-selection settings (minimum junction degree $g_{\min}$ and clustering radius ρ), communication settings (V2V/V2I range and building obstruction), and the maximum hop count H . In the case study the interface used a V2V/V2I range of 300 m, a 120 s snapshot interval, $g_{\min} = 4$, and $\rho = 20$ m as defaults.

3.2. Candidate RSU Selection

SUMO can produce a large set of junctions, including intermediate road points, internal nodes, and closely located intersections; using all of them as candidates enlarges the search space and adds nearly equivalent positions. The platform therefore applies a two-stage reduction: it keeps only non-internal junctions of degree at least $g_{\min}$, then sorts them by decreasing degree and runs greedy spatial clustering, accepting a node only when its distance to every accepted candidate exceeds ρ . Because higher-degree nodes are processed first, they represent their local groups.

Algorithm 1: Candidate RSU filtering by degree and greedy spatial clustering

Input: Junction set J , non-internal edge set E , minimum degree $g_{\min}$, clustering radius ρ

Output: Candidate RSU set C

```

1 Initialize  $\deg(j) \leftarrow 0$  for all  $j \in J$ ;
2 foreach  $(u, v) \in E$  do
3   | increment  $\deg(u)$  and  $\deg(v)$ ;
4  $F \leftarrow \{j \in J : \deg(j) \geq g_{\min}\}$ ;
5 Sort  $F$  by non-increasing degree;  $C \leftarrow \emptyset$ ;
6 foreach  $j \in F$  do
7   | if  $C = \emptyset$  or  $\min_{c \in C} \text{dist}(j, c) > \rho$  then
8     | |  $C \leftarrow C \cup \{j\}$ ;
9 return  $C$ ;
  
```

This is a preprocessing heuristic, not an optimal RSU-placement algorithm. Its purpose is to build a manageable candidate set before the deployment model is solved. Since it may drop a location that would later be useful, the sensitivity of the retained set to $(g_{\min}, \rho)$ is reported in Section 4 instead of hiding these choices as fixed constants.

3.3. Direct and Multi-Hop Connectivity

For each snapshot $s \in \mathcal{S}$, let $\mathcal{V}_s = \{v_1, \dots, v_{n_s}\}$ be the active vehicles and $\mathcal{R} = \{r_1, \dots, r_m\}$ the candidate RSUs. The binary V2V matrix $A_s \in \{0, 1\}^{n_s \times n_s}$ has $(A_s)_{ij} = 1$

when v_i and v_j are within the V2V range and no building polygon blocks the segment joining them; under the bidirectional-link model it is symmetric. The binary V2I matrix $B_s \in \{0, 1\}^{n_s \times m}$ has $(B_s)_{ik} = 1$ when v_i can directly reach r_k under the V2I range and LoS. Obstruction uses a 2-D segment–polygon intersection test, so the result is a geometric binary approximation; it does not model attenuation, fading, interference, or packet delivery.

A vehicle without direct V2I access may still reach an RSU through other vehicles. Define the augmented adjacency matrix $\tilde{A}_s = \beta(A_s + I)$, where $\beta(X)$ maps every positive entry of X to one and every zero entry to zero. Cumulative connectivity is

$$R_{s,1} = B_s, \quad R_{s,h} = \beta(\tilde{A}_s R_{s,h-1}), \quad h = 2, \dots, H. \quad (1)$$

The identity term keeps the paths already found at smaller hop counts, which gives $R_{s,1} \leq R_{s,2} \leq \dots \leq R_{s,H}$ elementwise. The first-arrival matrices are $S_{s,1} = R_{s,1}$ and $S_{s,h} = R_{s,h} - R_{s,h-1}$ for $h \geq 2$.

Proposition 1. For any snapshot s , $(R_{s,h})_{ik} = 1$ if and only if there is a path from vehicle v_i to candidate RSU r_k using at most h communication hops, where the final hop is V2I and every previous hop is V2V.

Proof. By induction on h . For $h = 1$, $R_{s,1} = B_s$, so an entry is one exactly for a direct V2I link. Assume the statement holds for $h - 1$. The entry $(\tilde{A}_s R_{s,h-1})_{ik}$ is positive if there is an index j with $(\tilde{A}_s)_{ij} = 1$ and $(R_{s,h-1})_{jk} = 1$. If $j = i$, the identity term keeps a path of at most $h - 1$ hops. If $j \neq i$, then $(A_s)_{ij} = 1$ adds one V2V hop from v_i to v_j before a path of at most $h - 1$ hops from v_j to r_k . Conversely, every path of at most h hops either already has at most $h - 1$ hops or begins with one V2V hop followed by such a path. Binarization therefore gives exactly the stated reachability relation. $\square$

Corollary 1. For $h \geq 2$, $(S_{s,h})_{ik} = 1$ if and only if the minimum number of hops from v_i to r_k is exactly h ; for $h = 1$, $S_{s,1} = B_s$ identifies the direct connections.

The corollary follows from Proposition 1 and the monotonicity of $R_{s,h}$: each reachable vehicle–RSU pair appears for the first time at a single hop value only. Figure 2 shows a compact example, where the purple cells mark the first arrival at the corresponding hop. Equation (1) uses dense matrix multiplication followed by thresholding; a Boolean-semiring implementation could reduce the average effort without changing the dense worst-case order, and is left as future work.

R_3	r_1	r_2	r_3	S_2	r_1	r_2	r_3	S_3	r_1	r_2	r_3
v_1	1	1	1	v_1	0	0	1	v_1	0	1	0
v_2	1	1	1	v_2	1	1	0	v_2	0	0	0
v_3	1	1	1	v_3	0	0	1	v_3	1	0	0
v_4	0	0	0	v_4	0	0	0	v_4	0	0	0

Figure 2. Illustrative cumulative (R_3) and first-arrival (S_2, S_3) matrices for $H = 3$. Purple cells mark the hop at which each vehicle–RSU pair first becomes reachable; vehicle v_4 reaches no real RSU within the hop limit.

3.4. Optimization-Oriented Export and Cardinality

Let $K_s = \sum_{h=1}^H \text{nnz}(S_{s,h}) = \text{nnz}(R_{s,H})$ be the number of reachable real vehicle–RSU pairs in snapshot s . The export module creates one tuple for each real pair at its minimum hop and one artificial no-service tuple for every active vehicle:

$$\text{CVR}_s = \{(s, h, i, k) : (S_{s,h})_{ik} = 1\} \cup \{(s, H + 1, i, r_\infty) : i \in \mathcal{V}_s\}.$$

The artificial RSU r_∞ (identifier 0) is not only physical disconnection: it is the optimization option of leaving a vehicle unserved when real candidates are not deployed, are saturated, or are rejected by cost or capacity constraints. It is therefore included even when the vehicle can reach real RSUs, and is penalized at hop $H + 1$. The cardinality is $|\text{CVR}_s| = K_s + n_s$, with $n_s \leq |\text{CVR}_s| \leq n_s(m + 1)$ for $m \geq 1$; the hop limit H does not multiply the real tuples, since each reachable pair is exported once, at its minimum hop. The dataset can be written to an OPL/.dat file or kept in memory for other solvers.

4. Experimental Results

4.1. Experimental Settings

The evaluation uses one real scenario in the Historic Centre of Quito, Ecuador. The goal is not to compare cities but to measure how the candidate-generation parameters change the size of the optimization search space while the road network stays fixed. The two factors are the minimum junction degree $g_{\min}$ and the greedy clustering radius ρ . The imported network has 1211 eligible junctions; the SUMO run produced 31 snapshots (one every 120 s) with 132 vehicles and a 300 m range. All configurations reuse the same network and node-exclusion rule, which isolates the effect of $(g_{\min}, \rho)$. The study area (Figure 3) corresponds to an OpenStreetMap extract spanning about -78.535° to -78.497° longitude and -0.266° to -0.211° latitude (UTM zone 17S, WGS84), downloaded on 5 June 2026. The pipeline ran with SUMO 1.26.0 and Python 3.14 (NumPy 2.4, pandas 3.0) on Windows 11 (x86-64); vehicle demand was generated with `randomTrips.py` using its default seed (no `--random` flag), so the mobility is reproducible.

4.2. Results

Table 1 reports, for four configurations, the retained candidates Y and the reduction $100(1 - Y/X)$ over the $X = 1211$ eligible junctions. The trend is monotonic and matches the design of the heuristic: raising $g_{\min}$ drops low-degree junctions before clustering, and raising ρ removes candidates near an already retained higher-degree node. The permissive setting (A) keeps 414 candidates (65.8% reduction), while the restrictive setting (D) keeps only 177 (85.4%). The reference setting (C, $g_{\min} = 4$, $\rho = 20$ m) retains 333 candidates, about a quarter of the eligible junctions; Figure 3 maps these candidates over the study area.

Table 1. Candidate-set sensitivity in the Historic Centre of Quito ($X = 1211$ eligible junctions). Values were produced by running the filtering code on the same network.

Config.	$g_{\min}$	ρ (m)	Candidates Y	Reduction (%)
A	3	15	414	65.8
B	3	20	386	68.1
C	4	20	333	72.5
D	5	25	177	85.4

This matters because binary deployment variables and many assignment or routing structures grow with the number of candidate facilities. A smaller set is not automatically better, though: it speeds up the model but may drop useful locations, so the reduction is a preprocessing trade-off, not proof of a better deployment. On the same scenario the platform completed the full transformation from snapshots to matrices and tuples, producing 1390 direct V2I contacts across the 31 snapshots for the 132 vehicles and the 333 reference candidates.

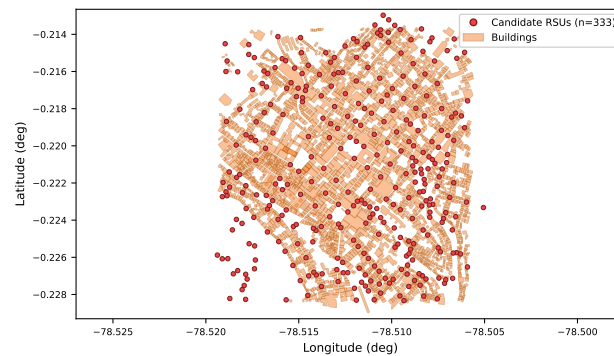


Figure 3. Candidate RSUs produced by the platform for the reference configuration (C , $g_{\min} = 4$, $\rho = 20$ m; $n = 333$, red) over the building footprints (orange) of the Historic Centre of Quito, in geographical coordinates.

Three limitations remain: the binary geometric link model ignores path loss, fading, interference, channel load, and packet reliability; each snapshot is processed independently, so the recurrence describes paths *inside* a snapshot, not time-respecting store-carry-forward routes; and candidate filtering is heuristic, with its effect on a later deployment objective still unquantified.

5. Conclusions and Future Work

This paper presented a web platform that turns a geographical area and SUMO mobility output into optimization-ready V2V/V2I connectivity data, combining candidate RSU generation, direct-link evaluation with building obstruction, cumulative and minimum-hop matrix processing, traceable identifiers, and export. A formal result shows that the recurrence finds every vehicle–RSU relation reachable within the hop limit, and the first-arrival matrices assign each real pair its minimum hop; the exported set adds one artificial no-service option per vehicle, an optimization decision rather than mere disconnection. On the Historic Centre of Quito, filtering keeps 177–414 of 1211 eligible junctions depending on $(g_{\min}, \rho)$, a preprocessing trade-off that shrinks the model without guaranteeing a better deployment. Future work includes comparing the recurrence against breadth-first search, sparse and Boolean-semiring kernels, dynamic tuple generation inside the optimization method (e.g. decomposition or column generation), richer propagation and reliability models, and measuring the effect of filtering on deployment cost, coverage, capacity, and solution time.

Author Contributions: Conceptualization, methodology, software, validation, writing—original draft preparation, and writing—review and editing, E.X.E.S. The author has read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The source code and the generated datasets (SUMO network, snapshots, matrices, and exported tuples) are available in the project repository.

Acknowledgments: During the preparation of this manuscript, an AI-based assistant was used for LaTeX formatting and draft editing; the author reviewed and edited the content and takes full responsibility for the publication.

Conflicts of Interest: The author declares no conflict of interest.

References

1. Urquiza-Aguiar, L.; Tripp-Barba, C.; Aguilar-Igartua, M. A Stochastic Optimization Model for the Placement of Road Side Units. In *Proceedings of ADHOC-NOW*; Springer: Cham, Switzerland, 2016.
2. Lopez, P.A.; Behrisch, M.; Bieker-Walz, L.; et al. Microscopic Traffic Simulation using SUMO. In *Proceedings of the 21st IEEE International Conference on Intelligent Transportation Systems*; IEEE: Maui, HI, USA, 2018; pp. 2575–2582.
3. OpenStreetMap contributors. OpenStreetMap. Available online: <https://www.openstreetmap.org> (accessed on 20 July 2026).
4. Streamlit Inc. Streamlit. Available online: <https://streamlit.io> (accessed on 20 July 2026).
5. Python Visualization. Folium: Python Data, Leaflet.js Maps. Available online: <https://python-visualization.github.io/folium/> (accessed on 20 July 2026).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.